🥇

Riassunto modulo 1

Image Processing vs Computer Vision vs Computer Graphics

> 💡 **Image Processing:** campo che si occupa di migliorare le immagini regolandone i numerosi parametri e caratteristiche
> → è considerato pertanto un sottoinsieme della **Computer Vision**

→ a un'immagine in input vengono applicate delle **trasformazioni** per restituire un'immagine risultante in output → le trasformazioni agiscono, ad esempio, sulla nitidezza *(sharpening)* o sulla messa a fuoco *(smoothing)*

> 💡 **Computer Vision:** si occupa di istruire i computer o le macchine per acquisire una comprensione ad alto livello a partire da immagini digitali o video in ingresso, con lo scopo di automatizzare i compiti che il sistema visivo umano è in grado di svolgere

→ impiega numerose tecniche, e l'Elaborazione delle Immagini (*Image Processing*) è solamente una di queste

> 💡 **Computer Graphics:** si occupa di creare o manipolare un insieme di immagini, dove per immagine si intende un disegno o un qualsiasi tipo di sketch in grado di rappresentare informazioni significative sotto forma di figure visive

→ la grafica computerizzata è ampiamente utilizzata nel campo dell'informatica e del software quando vi è un insieme di immagini che deve essere creato o manipolato, come ad esempio nei film digitali, nell'industria dell'intrattenimento o nella fotografia digitale

Immagini digitali

Un'**immagine** è una **funzione continua** che rappresenta una scena reale

$$f : \Omega \subset R^2 \rightarrow R$$

Un'**immagine *discreta* (o *digitale*)** è una **matrice** di dimensione **MxN** ottenuta dalla **discretizzazione** della funzione f

→ come un'immagine è rappresentata da una matrice, analogamente ogni matrice può essere rappresentata come un'immagine (può essere anche priva di significato)

Per ogni pixel abbiamo 3 valori: **RGB** → 3 canali: una scala di rossi, una di verdi e una di blu

→ l'immagine è ottenuta mostrando sullo schermo il risultato della combinazione di questi 3 canali per determinare il colore finale del pixel, per ogni pixel dell'immagine

Esistono diversi formati per memorizzare le immagini

Inoltre, alcune immagini per scopi specifici possono avere formati specifici → es. **medical images** (TIFF, DICOM, NIfTI)

→ questo permette di memorizzare **metadati** speciali per il caso d'uso (es. per le immagini mediche può essere utile avere come metadati alcune informazioni del paziente) ⇒ la prima parte del file contiene i metadati, la seconda l'immagine

Leggere immagini in Python

Esistono 4 librerie principali per leggere e/o manipolare immagini in Python:

- OpenCV → Libreria utilizzata principalmente per computer vision e fast processing
- Matplotlib → Libreria per visualizzazione e plotting
- Pillow → Libreria utilizzata per modificare le immagini dal punto di vista più fotografico
- ImageIO → Libreria utilizzata per veloce lettura e scrittura di diversi formati di immagini e video

Image compression

💡 La compressione di un'immagine consiste nel ridurre la dimensione dell'immagine, cioè ridurre il numero di bit utilizzati per memorizzare l'immagine

💡 **Lossless compression:** compressione in cui non vengono perse informazioni
→ l'algoritmo di compressione permette di ricostruire, a partire dall'immagine compressa, le informazioni complete dell'immagine iniziale

💡 **Lossy compression:** compressione in cui si ha una perdita di alcune informazioni
→ non è possibile ricostruire l'immagine iniziale

Tipi di immagini digitali

- Immagini fotografiche → In scala di grigi o a colori
- Immagini mediche → Solitamente in scala di grigi, dove l'intensità dei pixel rappresenta la densità del materiale (più scuro = più denso) o il coefficiente di attenuazione, che è comunque proporzionale alla densità
- Immagini al microscopio → Possono essere diverse, essendoci molte tipologie diverse di microscopio
- Immagini termiche → Sono immagini a 1 canale rappresentate come una mappa colore, dove l'intensità dei pixel rappresenta la temperatura (blu = freddo, magenta = caldo)
- Immagini iperspettrali → in 4 dimensioni: una (o più) dimensione aggiuntiva per immagazzinare informazioni aggiuntive

Image Processing Tools

Pixel processing

Il task più semplice da realizzare su un'immagine è applicare la stessa trasformazione ad ogni pixel dell'immagine, modificandoli tutti *(pixel to pixel)*

→ la trasformazione è **indipendente** per ogni pixel

Istogramma di un'immagine

💡 L'istogramma di un'immagine è un grafico che mostra come l'intensità dei pixel è distribuita nell'immagine → in particolare, mostra quanti pixel hanno un determinato valore di intensità

- Asse y: numero di pixel
- Asse x: valori di intensità da 0 a 255
  - immagini più scure si concentrano a sinistra
  - immagini più chiare presentano valori maggiori a destra
  - una distribuzione normale (a campana) significa che l'immagine è bilanciata

Gli istogrammi sono utilizzati per diversi scopi:

- aggiustamenti del contrasto
- molte task di preprocessing per computer vision
- segmentazione (molto basica) → si fissa una soglia di intensità e si ritaglia in questo modo una parte di istogramma, identificando in questo modo un oggetto
- equalizzazione → ridistribuzione dell'intensità dei pixel in maniera uniforme e bilanciata

  L'equalizzazione è un'operazione molto basica, che migliora il contrasto di un'immagine (ma ci sono algoritmi di processing che funzionano meglio)

Image noise

💡 **Image noise:** è una variazione casuale non desiderata nei valori dei pixel che non corrisponde alla scena reale

→ può provenire dal sensore fotografico, da errori di trasmissione, dalla statistica dei fotoni (nelle immagini a raggi X si utilizzano i fotoni per creare l'immagine)

Esempi di distribuzione statistica di *noise* ***non correlata:***

- **Noise Gaussiana** con media zero, anche detta *white noise* → è una noise **additiva**

  $$x_n = x + \epsilon$$

  dove $\epsilon \simeq N(0, \sigma)$

  → è la più comune, dovuta alla digitalizzazione dell'immagine o al sensore fotografico
- **Noise uniforme** → distribuzione uniforme
- **Noise esponenziale** → istogramma ha una forma che segue funzione esponenziale
- **Noise impulsiva**, nota anche come *salt and pepper noise* → alcuni pixel sono randomicamente 0 o 1

Esempi di distribuzione statistica di *noise* ***correlata****:*

- **Noise di Poisson**, in cui la varianza dipende dall'intensità del pixel

  → presente nelle immagini mediche
- **Noise di Speckle**

  $$x_n = x + x * \epsilon$$

Image Filtering

Un *filtro* è un tipo di processing *pixel to pixel*

Possiamo distinguere 3 classi di filtri:

- **spaziali** *(image filtering)* → agisce nello spazio dell'immagine, pixel to pixel
- **temporali** *(temporal filtering)* → si applica ad una sequenza di immagini, modificando i pixel
- **sulla frequenza** (filtri più comuni) → non si applica allo spazio dell'immagine, ma allo spazio della frequenza (utilizzando **transformer**)

Matematicamente, un filtro (di qualsiasi classe) è una funzione o un operatore H che trasforma un segnale in input in un segnale in output

$$y = H(x)$$

Agiscono:

- filtrando (bloccando/attenuando) alcune caratteristiche dell'immagine, non desiderabili
- lasciando invariate altre caratteristiche, desiderabili

Linear Shift Invariant Systems (LSIS)

I filtri lineari si basano sull'assunzione che l'image processing può essere modellato tramite un *Linear Shift Invariant System*, la funzione H vista sopra:

$$f(x) \rightarrow \text{LSIS} \rightarrow g(x)$$

In questo caso, *ogni pixel* viene modificato nella *stessa maniera* (linearità nel filtro vuol dire che si applica la stessa funzione a tutti i pixel)

> 💡 Un segnale è una funzione a canale singolo
>
> $$f : \mathbb{R} \rightarrow \mathbb{R}$$
>
> → è un sottoinsieme dell'immagine: prima applichiamo la funzione al segnale (perché più semplice), poi a tutta l'immagine

Proprietà dei LSIS:

- **Linearità**

$$f_1 \rightarrow \text{LSIS} \rightarrow g_1 \quad f_2 \rightarrow \text{LSIS} \rightarrow g_2$$
$$\alpha f_1 + \beta f_2 \rightarrow \text{LSIS} \rightarrow \alpha g_1 + \beta g_2$$

- **Shift invariance** (invarianza alla traslazione o invarianza spaziale)

$$f(x) \rightarrow LSIS \rightarrow g(x)$$
$$f(x-a) \rightarrow LSIS \rightarrow g(x-a)$$

Perché i LSIS sono importanti per le immagini?

Spesso le immagini sono ottenute catturandole attraverso lenti

→ essendo l'oggetto lontano dalla lente al momento della cattura, è come se l'immagine catturata fosse ottenuta applicando un LSIS all'oggetto iniziale → la lente si comporta come LSIS

Questo implica che l'acquisizione delle immagini ha delle proprietà (*linearità* e *shift invariance*) che possono essere sfruttate per ricostruire l'esatta scena a partire dall'immagine catturata, recuperando le informazioni perse

Convoluzioni

La **convoluzione** è un esempio di LSIS, ovvero è un **filtro lineare**, in cui la funzione H è una convoluzione

La convoluzione coinvolge due funzioni:

- f → l'immagine da modificare
- h → il **kernel** della convoluzione, che rappresenta la trasformazione da applicare all'immagine

→ la convoluzione si indica con un *

$$g(x) = f(x) * h(x) = \int_{-\infty}^{\infty} f(\tau) \cdot h(x - \tau)\, d\tau$$

Quando si applica una convoluzione, il risultato è una funzione che soddisfa **linearità** e **shift invariance** ⇒ la convoluzione è un **LSIS**

L'unità (o elemento neutro) dell'operazione di convoluzione è la *unit impulse function*

$$\delta(x) = \begin{cases} 1/2\varepsilon, & |x| \leq \varepsilon \\ 0, & |x| > \varepsilon \end{cases} \qquad \varepsilon \to 0$$

Dal punto di vista geometrico, poiché i termini si annullano nel calcolo dell'integrale, l'area sotto l'impulso è sempre costantemente pari a 1 ⇒ in una convoluzione tra l'impulso unitario e un'altra generica funzione b(x) l'impulso è in grado di "filtrare" o "estrarre" il valore esatto della funzione con cui viene convolto, senza alterarlo

Convoluzioni in 2D

Nel caso di funzioni in due dimensioni, la convoluzione si estende come segue:

$$\iint f(\tau, \mu) \cdot h(x - \tau, x - \mu)\, d\tau d\mu$$

Nel caso **discreto** (per le immagini) è necessario discretizzare le funzioni, rappresentando immagine e kernel come matrici

→ il *kernel* della convoluzione h è anche detto ***maschera* di convoluzione**, *filtro* di convoluzione

→ per avere un centro da sovrapporre *pixel-per-pixel*, il kernel è un quadrato di lato dispari

Quando il kernel viene applicato a un singolo pixel, prima viene invertito due volte (-m e -n) e poi viene effettivamente applicato sul pixel, sovrapponendo il centro del kernel al pixel e moltiplicando *pixel-a-pixel* i valori dell'immagine con quelli del kernel

Come gestire la condizione in cui si applica la convoluzione ai bordi? Diverse soluzioni per estendere l'immagine

- Ignorare il bordo
- Settare i pixel aggiuntivi a 0 (completamente neri)
- Ripetere i valori dell'immagine esternamente

Tipi di filtri lineari

- Unit impulse function → Eseguendo la convoluzione con la funzione impulso come kernel, otteniamo in output la stessa immagine di input
- Box filter → Utilizzando il *box kernel* (o filtro a scatola), con un'area quadrata con valore costante pari a uno, l'immagine in input appare saturata
- Fuzzy filter → In questo tipo di filtro, il valore massimo si trova in posizione centrale, mentre i valori dei blocchi circostanti decrescono simmetricamente mano a mano che si allontanano dal centro del filtro

  → l'immagine appare più regolare *(smooth)* e non sono visibili blocchi

  Matematicamente, il *fuzzy filter* può essere rappresentato tramite la funzione gaussiana

Filtri non lineari

Rimanendo nello spazio dell'immagine, possiamo avere anche filtri non lineari → la trasformazione applicata ai pixel differisce da pixel a pixel

→ essendo le convoluzioni dei LSIS, non possiamo usarle in questo caso

Perché utilizzare filtri non lineari? Es. per rimuovere la *salt and pepper noise* dall'immagine

- Median filter → Per ogni pixel:
  1. Si crea una maschera quadrata sovrapposta centralmente attorno al pixel
  2. Si ordinano i valori dei pixel nella maschera
  3. Si considera la mediana dei valori
  4. Si sostituisce la mediana calcolata al pixel centrale considerato
- Bilateral filter → Si basa sull'idea di utilizzare un kernel Gaussiano diverso per ogni pixel, tenendo conto del contesto in cui è inserito il pixel → si modella il filtro sulla base dei valori dei pixel vicini

  Confrontando il valore ottenuto con la gaussiana dei valori vicini con il valore del pixel possiamo eseguire determinate operazioni → es. possiamo eliminare i pixel che hanno un valore di luminosità troppo diverso dalla maschera

The Discrete Fourier Transform and its use in Image Processing

Numeri complessi

Fourier Transform coinvolge i numeri complessi

Qualsiasi numero complesso z può essere scritto come:

$$z = a + ib$$

- a → è la parte reale
- i b → è la parte complessa, in cui i è l'**unità immaginaria**

→ per rappresentare un numero complesso abbiamo bisogno di entrambe le informazioni

La **magnitudo** *(magnitude)* è il **modulo** del numero complesso:

$$|z| = \sqrt{a^2 + b^2}$$

→ geometricamente rappresenta la lunghezza del vettore da (0,0) a (a,b)

La **fase** *(phase)* è l'angolo che il vettore individuato dal modulo del numero complesso forma con l'asse positivo dei reali (ascissa):

$$\phi = arg(z) = arctan(\frac{a}{b})$$

Utilizzando magnitudo e fase possiamo riscrivere un numero reale z come:

$$z = |z|e^{i\phi}$$

→ applicando la formula di Eulero si ha che:

$$e^{i\phi} = cos(\phi) + i \cdot sin(\phi)$$

Fourier Theory

💡 Fourier afferma (dimostra) che ogni funzione periodica può essere riscritta come combinazione lineare di sinusoidi (seno e coseno)

$$f(x) = A\sin(2\pi ux + \phi)$$

- A → ampiezza
- phi → fase
- T → periodo
- u → frequenza (= 1/T)

L'idea è che, sommando le funzioni infinite volte, è possibile ottenere la funzione esatta

→ si ottiene così una rappresentazione di un segnale f(x) in termini di **ampiezza** e **fase** delle sinusoidi che lo costituiscono

*Partendo da un segnale periodico o non periodico, come trovare **ampiezza** e **fase?*** Con la **Trasformata di Fourier** F(u) di un segnale f

→ la Trasformata di Fourier è, di fatto, una **corrispondenza** dal dominio dello spazio al dominio della frequenza

Trasformata di Fourier

Ogni funzione (con alcune proprietà di regolarità) può essere approssimata da una somma infinita di funzioni seno e coseno con ampiezze e frequenze diverse:

$$f(x) = \sum_{k=1}^{N} A_k \cos(kx) + B_k \sin(kx)$$

→ questa somma è detta **somma (o serie) di Fourier**

Ogni funzione viene quindi rappresentata conoscendo le caratteristiche, ampiezza e frequenza, dei termini della somma che approssima la funzione → possiamo dire che la funzione esatta f(x) può essere rappresentata nel cosiddetto dominio della frequenza conoscendo i valori dei coefficienti (A_k, B_k)

Come trovare le componenti di Fourier Ak e Bk in base alla funzione da rappresentare? Tramite la cosiddetta **Trasformata di Fourier**, che calcola i coefficienti della serie di Fourier

La Trasformata continua di Fourier

💡 La **Trasformata (continua) di Fourier** F è una mappa che trasforma una funzione in un'altra funzione, operando tra due spazi di funzioni H e K, dove H è lo spazio contenente le *funzioni del dominio del tempo* f e K è lo spazio contenente le *funzioni del dominio della frequenza* $\hat{f}$

$$F : H \to K$$

dove

$$\hat{f}(w) = \int_{-\infty}^{\infty} f(x) e^{-i2\pi x} dx$$

La Trasformata (continua) inversa di Fourier è invece la mappa

$$F^{-1} : K \to H, \quad F^{-1}(\hat{f}) = f$$

tale che

$$F^{-1}(F(f)) = f$$

Qual è la relazione tra la Trasformata di Fourier e le sinusoidi?

$$e^{i\theta} = \cos\theta + i\sin\theta$$

dove i è l'unità immaginaria $i = \sqrt{-1}$

Perché la Trasformata di Fourier è un numero complesso? Perché necessita di memorizzare due componenti: l'**ampiezza** e la **fase**

La Trasformata discreta di Fourier

Quando scriviamo algoritmi non possiamo utilizzare rappresentazioni continue → si utilizza perciò la **discretizzazione** della Trasformata continua di Fourier

La Trasformata discreta di Fourier (DFT) è una funzione che va da uno spazio vettoriale reale a un altro spazio vettoriale complesso ⇒ il segnale continuo viene **discretizzato in un vettore**

- la componente reale rappresenta l'**ampiezza dell'onda coseno** nel dominio della frequenza
- la parte immaginaria corrisponde all'**ampiezza dell'onda seno**

→ insieme, queste componenti forniscono una rappresentazione completa delle caratteristiche in frequenza di un segnale

L'**algoritmo per il calcolo della DFT** è chiamato **Fast Fourier Transform (FFT)** e ha una complessità computazionale di $O(N \cdot \log_2(N))$ quando $N = 2^p$ (cioè quando **N è una potenza di 2**)

Comprendendo le componenti in frequenza di un segnale, possiamo eseguire diverse operazioni come il filtraggio, la ricostruzione del segnale e molto altro

Relazione tra magnitudine e fase con la Trasformata di Fourier:

- Magnitudine → "quanto" di una frequenza è presente
- Fase → "dove" è posizionata quella frequenza (struttura)

Perché:

- mantenere l'ampiezza ma randomizzare la fase ⇒ rumore di tipo texture

- mantenere la fase ma cambiare l'ampiezza ⇒ struttura per lo più preservata

  → questa è esattamente l'idea chiave alla base dell'analisi delle immagini di Fourier

Come leggere la DFT di un'immagine?

Le alte frequenze sono necessarie per creare i bordi (o contorni) degli oggetti

Il teorema di convoluzione

Consideriamo la convoluzione tra due funzioni f e h

> ❗ La convoluzione $f(x) * h(x)$ nel dominio spaziale (o temporale) equivale al prodotto $F(u) \cdot H(u)$ nel dominio della frequenza (o dominio di Fourier)
> → vale anche il contrario

→ questo è alla base di un algoritmo molto veloce per eseguire la convoluzione tra due funzioni: la convoluzione viene eseguita nel dominio della frequenza, con oggetti più piccoli (dal calcolo di un integrale ci si riconduce alla moltiplicazione di matrici)

Image filtering nel dominio della frequenza

I valori nel dominio di Fourier, solitamente chiamati frequenze, hanno alcune proprietà utili per la manipolazione delle immagini, come:

- *ridurre la noise nell'immagine (denoising)*

  Per eliminare il rumore da un'immagine utilizzando un filtro Fast Fourier Transform (FFT), possiamo utilizzare un metodo semplice che consiste nell'applicare un filtro *low-pass* nel dominio della frequenza

  → l'idea è di sopprimere il rumore ad alta frequenza, che in genere si trova nelle frequenze più alte dello spettro di Fourier, preservando al contempo le componenti a bassa frequenza, che generalmente corrispondono alla struttura importante dell'immagine

  Tuttavia, si ottiene un'immagine sfocata → Per mitigare gli *artefatti da risonanza* (sfocatura), è fondamentale **evitare transizioni eccessivamente brusche nel dominio della frequenza:** mentre un filtro *low-pass* "ideale" (una maschera circolare netta) tronca istantaneamente le alte frequenze provocando oscillazioni nel dominio spaziale, l'impiego di filtri a transizione graduale porta a un risultato più naturale.

  Al contrario, eliminando le frequenze più basse *(high-pass filtering)* si rimuovono le aree a luminosità costante e, aumentando l'intensità del filtro, si mettono in evidenza i bordi

- *ridurre lo spazio occupato dall'immagine (compressione)*

Computational Imaging and Inverse Problems

> 💡 Def (prof Piccolomini)
> **Computational Imaging** si riferisce a sistemi di acquisizione in cui i dati misurati non sono immagini vere e proprie, ma misurazioni indirette, codificate o incomplete che vengono trasformate in immagini attraverso algoritmi computazionali.

L'imaging computazionale è la formulazione della **formazione dell'immagine** come un **problema inverso**, in cui un'immagine o un oggetto incognito x viene stimato a partire dai dati misurati y invertendo un modello in avanti (o diretto):

$$y = \mathcal{F}x + e$$

→ la fisica dell'imaging è modellata dall'operatore $\mathcal{F}$

Problema inverso

> 💡 Def (Keller)
> Chiamiamo **due problemi l'uno l'inverso dell'altro** se la formulazione di ciascuno coinvolge in parte o del tutto la formulazione dell'altro

→ il problema che è studiato in modo più esteso è solitamente chiamato in avanti (o diretto), l'altro è chiamato inverso:

- Il problema **diretto** è solitamente orientato secondo un **rapporto causa-effetto**, nel senso che calcola l'effetto data la causa
- Il problema **inverso** è associato all'**inversione del rapporto causa-effetto** e consiste nel trovare le cause dati gli effetti

Nella fisica strumentale, il problema diretto è la **misurazione dell'oggetto**, ottenuta dallo strumento, mentre il problema inverso consiste nel **determinare l'oggetto** data la misurazione

> ❗ Nei problemi diretti di solito c'è una **perdita di informazione** dall'input all'output ⇒ pertanto, nella soluzione del problema inverso è **impossibile recuperare esattamente l'oggetto**, a causa dell'informazione persa in quello diretto

Problemi inversi lineari di imaging

I problemi inversi lineari di imaging sono caratterizzati da un **modello lineare** che descrive la relazione tra i **dati misurati** y e l'**oggetto incognito** x

Il modello lineare diretto può essere espresso come:

$$Ax = y$$

dove A è una matrice che rappresenta l'operatore lineare che agisce sull'immagine x per generare i dati y

La soluzione del problema inverso richiede di trovare l'**immagine esatta** x che rappresenta l'oggetto, dati i dati misurati y e la matrice A che rappresenta la fisica dello strumento di imaging

I dati sono solitamente affetti da ***noise*** (rumore), a causa di diversi processi fisici → possiamo considerare un rumore additivo, in modo che il modello diretto finale sia:

$$Ax = y + e$$

Esempi di applicazioni di imaging computazionale lineare:

- image denoise
- image deblur
- image superesolution
- image inpainting
- medical image reconstruction → tomografia computerizzata, risonanza magnetica

Mal-condizionamento dei problemi inversi

> 💡 Def
> Un problema è definito **ben posto** (*well-posed*) se:
>
> - esiste una soluzione per dati arbitrari
> - tale soluzione è unica
> - la soluzione dipende con continuità dai dati, ciò significa che varia poco a fronte di piccole variazioni dei dati

Il **problema diretto** è, in generale, ben posto (questo deriva da considerazioni fisiche)

> 💡 Un problema in cui almeno una delle tre condizioni definite precedentemente non è soddisfatta si dice **mal posto** (*ill-posed*)

I problemi inversi lineari di imaging sono mal posti

Risolvere il sistema lineare

$$Ax = y + e$$

è un compito numerico piuttosto semplice, ma nel caso di problemi inversi lineari mal posti, A è **mal condizionata** e la soluzione del sistema lineare diventa **difficile** per almeno uno dei tre seguenti motivi

Chiamiamo A la matrice che descrive il sistema di imaging e supponiamo che A sia di dimensione MxN

1. **Se M = N e A non è singolare** ⇒ il sistema ha un'unica soluzione, ma solitamente **non vi è dipendenza continua dai dati**

   → ciò significa che un piccolo cambiamento nei dati produce una grande variazione nella soluzione

2. **Se M > N** ⇒ il sistema lineare **non ha soluzioni** e al suo posto risolviamo il problema dei minimi quadrati:

   $$\min \|Ax - (y + e)\|_2^2$$

3. **Se M < N** ⇒ il sistema ha **infinite soluzioni possibili**

   → anche in questo caso, sostituiamo il sistema lineare con un problema di minimi quadrati

Il risultato è che l'inversione numerica produce risultati che sono fisicamente inaccettabili, anche se matematicamente accettabili

→ nei problemi inversi i dati sono sempre affetti da **noise** e il metodo di soluzione amplifica tale rumore, producendo una **funzione ampia e oscillante** che nasconde completamente la soluzione fisica corrispondente ai dati privi di noise

Anche se la soluzione del sistema lineare esiste ed è unica (caso 1), essa risulta **completamente corrotta** a causa del piccolo rumore sui dati (sistema mal condizionato), come si vede nell'immagine di seguito → la ***naive solution*** non è una soluzione buona né affidabile. Il rumore presente sui dati viene amplificato nella soluzione

The Model-Based Regularization Approach

Come curare la *ill-posedness*?

Il cattivo condizionamento non dipende dall'algoritmo, ma dai **dati**

Possiamo curare la malposizione (ill-posedness) cercando soluzioni approssimate che soddisfino vincoli aggiuntivi derivanti dalla fisica del problema.

→ l'informazione aggiunta è anche chiamata **informazione a priori** o **prior image information** in un contesto statistico, oppure **regolarizzazione** in un contesto matematico.

Tale informazione viene inserita mediante una o più funzioni di regolarizzazione, ciascuna pesata da una costante positiva chiamata **parametro di regolarizzazione**.

→ i metodi di regolarizzazione diretta calcolano la soluzione di un problema inverso mal posto (imaging) imponendo alcuni vincoli sulla soluzione stessa.

Il modello matematico risultante può essere impostato come un problema di:

- **minimizzazione non vincolata** (unconstrained minimization) → minimizzare una funzione F(x) senza vincoli imposti

$$\min_{x \in R^n} F(x)$$

- **minimizzazione vincolata** (constrained minimization) → minimizzare una funzione F(x) con alcuni vincoli

$$\min_{x \in D} F(x)$$

  → i vincoli più comunemente imposti riguardano l'appartenenza delle x a un intervallo

$$a_i \leq x_i \leq b_i \qquad\qquad x_i \geq 0 \quad \forall_i$$

Minimizzazione non vincolata (model-based)

Formulazione generale e più semplice di un metodo di regolarizzazione basato sul modello di tipo non vincolato (unconstrained):

$$\min_x d(Ax, y + \epsilon) + \lambda R(x)$$

dove:

- $d(Ax, y + \epsilon y^\delta)$ è una distanza nello spazio dell'immagine, anche detta *data fitting* perché misura la distanza tra i dati acquisiti (y) e il modello applicato alla soluzione calcolata (x)

  → è correlata alla distribuzione della noise ⇒ la funzione di distanza più diffusa è quella nel caso della noise Gaussiana (noise più comune):

$$d(Ax, y^\delta) = ||Ax - y^\delta||_2^2$$

- $\lambda R(x)$ è il termine di regolarizzazione, in cui λ è il **parametro di regolarizzazione** e R(x) è la **funzione di regolarizzazione**

  → più λ è grande, maggiore è l'importanza del termine di regolarizzazione (secondo termine) rispetto alla distanza del fitting (primo termine)

Questo approccio è detto ***model-based approach***, poiché l'informazione fisica rappresentata dalla matrice A è parte integrante del modello e anche la *noise* viene modellata all'interno del termine di data fitting

Funzioni di regolarizzazione

- 2 norm (Tichonov)

- 1 norm (sparsificatore)
- p norm (con p<1)

REGOLARIZZAZIONE DI TICHONOV

Nella **regolarizzazione di Tichonov** (regolarizzazione più classica), la funzione da minimizzare è una 2 norm

$$R(x) = ||x||_2^2 = \sum_{i=1}^{M}\sum_{j=1}^{N} x_{i,j}^2$$

Minimizzare la 2 norm di un vettore equivale a renderlo più uniforme (la regolarizzazione di Tichonov impone *smoothness* sul vettore dell'immagine) → aumentando λ, l'immagine viene resa più *smooth* (con la conseguente perdita di definizione nei bordi degli oggetti)

REGOLARIZZAZIONE SPARSIFICANTE (1 NORM)

Rappresentazione sparsa (sparse representation) significa che un segnale può essere rappresentato con pochi componenti significativi non nulli.

In questo caso, assumiamo che l'immagine di partenza sia anch'essa sparsa (altrimenti, se l'immagine iniziale non è sparsa, il risultato non sarà soddisfacente) → pertanto, la ricostruzione viene incentivata a contenere pochi pixel attivi

Questa tecnica è spesso associata all'imposizione di un **vincolo di non negatività** su x (x ≥ 0)

REGOLARIZZAZIONE P-NORM (0<P<1)

Le due funzioni di regolarizzazione precedentemente viste sono funzioni convesse

Un altro regolarizzatore che impone sparsità sul vettore immagine è il **p-norm, con 0 < p < 1** → questa funzione è *non convessa*

- Con p tendente a 0 si impone una sparsità maggiore
- Con p tendente a 1 si impone una sparsità minore

→ anche in questo caso, l'immagine di partenza deve essere sufficientemente sparsa

L'utilizzo più comune di questa funzione di regolarizzazione è con il **gradiente dell'immagine** come argomento (non direttamente il vettore immagine stesso)

Gradiente di un'immagine

Un'immagine è una funzione continua $f : R^2 \rightarrow R$ (che poi viene discretizzata in un vettore)

💡 Il **gradiente** di una funzione f (l'immagine) è una funzione

$$\nabla f : R^2 \rightarrow R^2$$

costituita dalle derivate parziali di f

$$\nabla f(x) = (\frac{\delta f}{\delta x}, \frac{\delta f}{\delta y})$$

Nel caso discreto, il gradiente è calcolato mediante apposite **formule di discretizzazione**

Il dominio del gradiente in un'immagine è particolarmente rappresentativo dell'immagine stessa, dal momento che rappresenta i **bordi** degli oggetti nell'immagine

Regolarizzatori nel dominio del gradiente

Possiamo applicare i regolarizzatori precedenti nel dominio del gradiente:

- 2 norm (Tichonov)
- 1 norm (Total Variation)
- p-norm (con 0 < p < 1)

REGOLARIZZAZIONE TOTAL VARIATION

La regolarizzazione TV (regolarizzazione a variazione totale) assume che le immagini siano **costanti a tratti** → questo si riflette in un **gradiente sparso**, ovvero un'immagine gradiente che presenta solo **pochi elementi non nulli**

La TV **penalizza** la somma dei gradienti, ovvero la quantità totale di gradienti:

- se il rumore è basso e l'immagine gradiente è sparsa, il termine di regolarizzazione TV(x) ha un valore piccolo
- al contrario, se il rumore è presente e il gradiente è meno sparso, il termine di regolarizzazione assume un valore più grande

Limite della regolarizzazione TV: tende a favorire soluzioni **costanti a tratti** *(staircasing effect)*

REGOLARIZZAZIONE 0-NORM

Regolarizzazione con p=0 è la regolarizzazione che impone maggiore sparsità sull'immagine (o sul gradiente, in base a cosa la applichiamo) → l'imposizione di un'elevata sparsità funziona bene quando vogliamo un'immagine che appaia come "a blocchi"

Il parametro di regolarizzazione

Come scegliere il valore del parametro di regolarizzazione?

Il parametro λ è il più critico da stabilire, non c'è un valore indicato:

- Un valore **troppo piccolo** produce un'immagine ricostruita rumorosa, mentre un valore **troppo grande** produce un'immagine con caratteristiche estreme imposte dal termine di regolarizzazione
- Esistono alcune regole per una scelta ottimale del parametro di regolarizzazione

  → tuttavia, esse richiedono solitamente molteplici esecuzioni del problema di minimizzazione per scegliere infine il valore migliore in base a determinati criteri predefiniti, ma processo richiede troppo tempo nelle applicazioni di imaging.

I criteri per la scelta di λ possono essere suddivisi in:

- Criteri che utilizzano una stima della noise e
- Criteri che non utilizzano alcuna informazione sulla noise e

PRINCIPIO DI DISCREPANZA

Il principio di discrepanza utilizza una **stima del rumore**

Lavora a posteriori: bisogna risolvere il problema di minimizzazione più volte e scegliere la soluzione migliore → costoso, ma è un metodo semplice (quindi molto usato in pratica)

→ è un metodo che funziona bene, ma tende a **sovrastimare** λ

SCELTA EURISTICA

Un modo pratico per scegliere il parametro di regolarizzazione è il metodo per tentativi ed errori

In molti casi, è possibile creare una **simulazione** con dati simili a quelli reali e utilizzare tale simulazione per selezionare un valore adeguato del parametro di regolarizzazione λ, da utilizzare poi sui dati reali.

Se tracciamo un grafico dell'errore relativo rispetto a λ, otteniamo una curva → scegliamo il minimo corrispondente al valore di λ che fornisce la migliore ricostruzione in termini di **errore relativo minore**

Pills of Numerical Optimization

Sulla base delle proprietà della funzione F posso scegliere diversi metodi per risolvere il problema di minimizzazione

Una proprietà importante della funzione F è la **derivabilità** → importante per l'applicabilità del metodo di **discesa del gradiente**

💡 Def
Una funzione f è **derivabile in un punto** $x_0 \in R^n$ se esiste una funzione $J : R^n \to R$ tale che:

$$\lim_{h \to 0} \frac{f(x_0 + h) - f(x_0) - J(h)}{h} = 0$$

💡 Def
Una funzione $f : R^n \to R$ è **convessa** se:

$$f(\alpha x + (1 - \alpha)y) \leq \alpha f(x) + (1 - \alpha)f(y), \quad \forall x, y \in R^n, \alpha \in R^+$$

f è **strettamente convessa** se:

$$f(\alpha x + (1 - \alpha)y) < \alpha f(x) + (1 - \alpha)f(y), \quad \forall x, y \in R^n, \alpha \in R^+$$

💡 Def
Sia $F : R^n \to R$
x_0 è strettamente un **minimo locale** se esiste $\epsilon > 0$ tale che:

$$f(x_0) < f(x) \ \ \forall x \in S(x_0, \epsilon)$$

x_0 è strettamente un **minimo globale** se:

$$f(x_0) < f(x) \ \ \forall x \in R^n$$

Quando minimizziamo una funzione vogliamo trovare il suo **minimo globale** (se esiste)

→ ma per vincoli matematici è difficile trovare un minimo globale quando ci sono molti minimi locali: l'algoritmo converge a un minimo locale e non sempre sappiamo se tale punto è anche un minimo globale

Condizioni ottimali per la minimizzazione non vincolata

Condizioni del primo ordine

💡 Prop
Se f è derivabile e x^* è un punto di minimo locale, allora

$$\nabla f(x^*) = 0$$

Se la funzione f è convessa si hanno alcune particolari proprietà riguardo la sua ottimizzazione:

importanza della convessità x problemi di minimizzazione

- se f è convessa, ogni minimo locale è un minimo globale
- se f è *strettamente* convessa, ha un unico minimo globale
- se f è convessa e derivabile, ogni punto stazionario di f è un minimo globale

Algoritmi di ottimizzazione

Tutti gli algoritmi di ottimizzazione sono algoritmi **iterativi**

Dato un **iterato iniziale** x_0, l'algoritmo calcola una sequenza di iterati $x_1, x_2, \dots$ tale che:

$$\lim_{k \to \infty} x_k = x^*$$

dove x^* è in genere un **minimo locale** (non il minimo globale)

→ solamente nell'ottimizzazione convessa x^* è certamente il minimo globale

L'implementazione di un metodo di ottimizzazione iterativo richiede di definire un **criterio di arresto**, ovvero una regola utilizzata per interrompere l'iterazione affinché non continui all'infinito

→ i criteri di arresto sono solitamente definiti mediante **disuguaglianze** che coinvolgono **tolleranze** definite dall'utente e legate alla precisione dell'algoritmo

Utilizzando le condizioni del primo ordine possiamo fissare la seguente regola:

$$||\nabla f(x_k)|| < \tau$$

dove τ è la tolleranza

→ quando la condizione è soddisfatta, l'algoritmo si arresta e l'ultimo iterato calcolato x_k è la soluzione restituita

Una caratteristica importante del metodo iterativo è la **velocità di approssimazione** della soluzione, cioè quante iterazioni necessita per soddisfare la condizione di arresto

→ euristicamente, è tale velocità legata al comportamento della curva di errore

Minimi quadrati

$$\min_x ||Ax - y||_2^2$$

La minimizzazione può essere risolta utilizzando un metodo iterativo come il **Conjugate Gradient Least Squares (CGLS)** applicato alle equazioni normali:

$$A^T A x = A^T y$$

→ con questo metodo è possibile eseguire la cosiddetta *regolarizzazione iterativa*

Regolarizzazione iterativa

Consiste nell'interrompere il metodo iterativo che risolve il problema di data fit:

$$\min_x ||Ax - y^\delta||_2^2$$

→ se riusciamo a fermarci al punto minimo della curva, otteniamo una buona immagine

Regolarizzazione di Tichonov

La funzione da minimizzare è nuovamente strettamente convessa e il metodo CGLS può essere applicato alle sue equazioni normali

In questo caso, se il parametro di regolarizzazione λ è impostato correttamente, non si verifica il fenomeno della **semi-convergenza**, ma il grafico dell'errore decresce verso lo zero

Regolarizzazione Total Variation

La funzione TV è **convessa ma non derivabile** in tutto il dominio (in particolare, in x = 0) ⇒ solitamente è approssimata da una funzione derivabile, ottenuta aggiungendo un parametro β positivo molto piccolo

Se si considera la funzione TV originale, è necessario un algoritmo di minimizzazione che non calcoli il gradiente della funzione

→ si utilizzano gli **algoritmi di prossimità,** che sostituiscono il gradiente di una funzione g con l'**operatore di prossimità** di g

Esistono molti algoritmi di prossimità → utilizzeremo l'**algoritmo di Chambolle-Pock** per risolvere la regolarizzazione TV

Regolarizzazione Total p-Variation

La regolarizzazione con la p-Variazione Totale (0 < p < 1) sul gradiente dell'immagine minimizza una funzione non differenziabile nello zero e **non convessa**

→ l'**algoritmo di Chambolle-Pock** può essere adattato per risolvere minimizzazioni non convesse

> ❗ Notare che, poiché la funzione è non convessa, **abbiamo più di un minimo locale** e l'algoritmo di ottimizzazione converge a uno di questi minimi locali, che in generale non è il minimo globale
> → **l'iterata iniziale x_0 è molto importante**, poiché il punto di convergenza dipende da x_0, nel senso che il percorso delle iterate calcolate x_k dipende da quello di partenza.

Image Deblur or Image Deconvolution

Data un'immagine sfocata e rumorosa, ricostruire un'immagine il più possibile vicina a quella che rappresenta l'oggetto (detta "immagine esatta" o "ground truth")

L'immagine sfocata può essere determinata da:

- un obiettivo con imperfezioni (quando l'immagine è fuori fuoco)
- il movimento relativo tra l'oggetto e il sistema di imaging (quando la sfocatura ha una direzione specifica)

→ in tutti i casi, l'immagine sfocata è ottenuta da una **convoluzione** dell'immagine esatta con un nucleo (*kernel*) di sfocatura, chiamato **Point Spread Function (PSF)**

L'obiettivo è, data una convoluzione rumorosa e supponendo di conoscere il nucleo (*kernel*) di convoluzione, **deconvolvere** l'immagine osservata per ottenere la *ground truth*.

→ questo è un tipico problema inverso lineare che può essere modellato come:

$$y^\delta = K * x + e$$

Il **LSIS** (*Linear Shift-Invariant System*) è descritto come una convoluzione → è possibile scrivere la convoluzione anche come un prodotto matrice-vettore

Space invariant blur

- L'immagine osservata è il dato acquisito dallo strumento e degradato da sfocatura e noise

- L'immagine originale rappresenta la *ground truth* che intendiamo ricostruire
- La Point Spread Function è il nucleo di convoluzione (kernel) che descrive come il sistema diffonde la luce (e quindi le imperfezioni) di un singolo punto
- La noise è la componente stocastica che contamina i dati e rende l'inversione naive instabile

In un sistema ottico, la PSF rappresenta la deformazione su ogni pixel dovuta alle imperfezioni della lente → Considereremo solo PSF **invarianti nello spazio** (*Space Invariant PSF*): ciò significa che la stessa PSF (lo stesso nucleo di convoluzione) viene applicata a ogni pixel dell'immagine esatta ⇒ l'immagine degradata finale è la somma delle immagini ottenute applicando la PSF a ciascun pixel

Deconvoluzione come problema inverso

Sappiamo che la soluzione ingenua (*naive*) del problema inverso si ottiene invertendo il modello lineare $K * x = y^\delta$ o $Ax = y^\delta$ per mezzo dei **minimi quadrati**

$$\min_x \|K * x - y^\delta\|_2^2$$

Come risolvere questo problema? Nel deblurring, il metodo computazionalmente più efficiente è utilizzare la **Trasformata di Fourier Discreta** (DFT)

- CASO SENZA NOISE

  Supponiamo che $f'(x, y)$ sia l'immagine restaurata (la chiamiamo f' e non f perché potrebbe non essere possibile ricostruirla *perfettamente*)

  Il modello di formazione dell'immagine restaurata (stima) è definito dalla convoluzione:

  $$f(x, y) * h(x, y) = g(x, y)$$

  Grazie al **teorema di convoluzione**, la relazione tra le DFT (*Discrete Fourier Transforms*) nello spazio delle frequenze diventa un semplice prodotto algebrico:

  $$F'(u, v) \times H(u, v) = G(u, v)$$

  > ❗ Ogni volta che dobbiamo eseguire la moltiplicazione matrice-vettore Av o $A^T w$, lo facciamo nello **spazio di Fourier**, poiché nel deblurring questa operazione corrisponde a una **convoluzione**

  → di conseguenza, per ottenere F'(u, v) eseguiamo la divisione punto a punto:

  $$F'(u, v) = \frac{G(u, v)}{H(u, v)}$$

  Infine, applicando la trasformata inversa (IDFT), otteniamo l'immagine nel dominio spaziale:

  $$f'(x, y) = IDFT(F'(u, v))$$

- CASO CON NOISE

  Il modello di acquisizione reale (con l'introduzione della **noise**) è:

  $$f(x, y) * h(x, y) + \eta(x, y) = g(x, y)$$

  La soluzione *naive* nel processo di deconvoluzione utilizza la **PSF h(x, y)** nota per cercare di ottenere l'immagine **Recuperata f'(x, y)**

Applicando la trasformata di Fourier otteniamo uno spettro dell'immagine estremamente luminoso e disturbato, che indica un'amplificazione eccessiva delle alte frequenze → Se si inverte il risultato, ovvero si calcola $f'(x,y) = IDFT(F'(u,v))$, si ottiene un'immagine completamente dominata dal rumore

Regolarizzazione

Dobbiamo sopprimere la noise tramite un'operazione di **regolarizzazione**, applicando approcci di regolarizzazione basati sul modello

$$\min_x ||K * x - y^\delta||_2^2 + \lambda R(x)$$

→ è anche possibile vincolare la soluzione, ad esempio imponendo la positività $(\min_{x \geq 0})$

Considerando la **regolarizzazione di Tichonov**, il problema diventa:

$$\min_x ||K * x - g||_2^2 + \lambda ||x||_2^2$$

→ si può calcolare la soluzione matematicamente, con l'aiuto di DFT

Se usiamo funzioni di regolarizzazione diverse da Tichonov, la soluzione si ottiene tramite **metodi di ottimizzazione iterativa** del problema di minimizzazione originale:

$$\min_z ||K * x - y^\delta||_2^2 + \lambda R(x)$$

Blind deconvolution

Fino ad ora abbiamo ipotizzato di conoscere la PSF (il kernel di sfocatura) → ma in alcuni casi reali la PSF non è nota: in questi casi parliamo di **blind deconvolution**

Dobbiamo recuperare sia il kernel che l'immagine ricostruita

→ il compito è molto più difficile e l'approccio basato sul modello non ha fornito risultati soddisfacenti ⇒ oggi questo compito viene affrontato mediante le **reti neurali** e i risultati sono di gran lunga migliori

Super resolution

La *super resolution* può avere diversi significati:

- aumento della risoluzione **spaziale** → nel caso di una sequenza temporale di immagini (es. video o sequenza di immagini mediche), consiste in un aumento di **frame**
- aumento della risoluzione **spettrale** → consiste nel migliorare la risoluzione di una **dimensione** specifica (es. frequenza, temperatura)
- aumento della risoluzione **spaziale** → consiste in un aumento del numero di **pixel** dell'immagine (aumento di dettaglio)

Consideriamo l'aumento della risoluzione spaziale

Nearest neighbour

L'algoritmo di super-risoluzione *nearest neighbour* è il metodo più semplice utilizzato per aumentare la risoluzione di un'immagine

In realtà, non crea nuovi dettagli: si limita a replicare i pixel esistenti

→ quando si ingrandisce un'immagine a bassa risoluzione, ogni nuovo pixel nell'immagine ad alta risoluzione assume il valore del pixel più vicino nell'immagine originale

$$I_{HR}(i,j) = I_{LR}\left(\text{round}\left(\frac{i}{s}\right), \text{round}\left(\frac{j}{s}\right)\right)$$

Pro:

- molto veloce
- molto semplice
- non introduce nuovi valori di pixel (si limita a replicare quelli esistenti)

Contro:

- produce immagini a blocchi o pixelate
- non ricostruisce i dettagli (non aggiunge informazione reale)
- qualità inferiore rispetto ad altri metodi di interpolazione (come il bilineare o il bicubico)

Interpolazione bilineare

Idea: il valore di ogni nuovo pixel viene calcolato come una media pesata dei 4 pixel più vicini → L'interpolazione è lineare in **due direzioni** (basata su un polinomio di grado 1):

- prima lungo la direzione x
- poi lungo la direzione y

→ ciascuno dei quattro pixel contribuisce con un peso **proporzionale alla sua distanza** dal nuovo punto: i pixel più vicini alla nuova posizione hanno pesi maggiori

Risultato:

- immagine più fluida rispetto al *nearest neighbour*
- artefatti a blocchi ridotti
- ancora leggermente sfocata

Interpolazione bicubica

L'interpolazione bicubica utilizza i **16 pixel circostanti** (un vicinato 4×4) invece di 4 → L'interpolazione si basa su **polinomi cubici** (di grado 3), che forniscono stime più fluide e accurate.

Risultato:

- immagini più nitide/fluide
- bordi più definiti
- meno sfocatura rispetto all'interpolazione bilineare

Super resolution come problema inverso

1. **Sfocatura Ottica B_k:** le imperfezioni ottiche o il movimento della camera sfocano l'informazione originale (PSF)
2. **Downsampling D:** il sensore della fotocamera ha un numero limitato di pixel ⇒ l'operatore D riduce la dimensione della matrice originale, scartando una grande quantità di dati
3. **Noise n_k:** infine, l'elettronica del sensore aggiunge un rumore casuale che contamina ulteriormente il segnale

$$y_k = DB_k x + n_k$$

La visione della super resolution come problema inverso consiste nell'individuare l'operatore di downsampling (oltre a rimuovere la noise) per applicare l'operazione inversa e fare *upscaling* dell'immagine, migliorandone la risoluzione

Operatori di downsampling

Esistono due modi principali in cui l'operatore di downsampling D può agire matematicamente per ridurre la risoluzione di un'immagine:

- **Zero-padding:** l'operatore agisce come un selettore di pixel, scegliendo un solo pixel per ogni blocco (ad esempio il primo in alto a sinistra) scartando tutti gli altri
- **Media:** ogni pixel dell'immagine a bassa risoluzione è la **media aritmetica** dei pixel del blocco corrispondente nell'immagine ad alta risoluzione

  → più vicino a come funzionano fisicamente i sensori delle fotocamere

Evaluation of Output Images

Mean Square Error (MSE)

$$MSE = \frac{1}{N}\sum_{i=1}^{N}(x_i - x_i^{GT})^2$$

→ **più basso** è il valore del MSE, **migliore** è l'immagine
→ tuttavia, il suo valore dipende dalla **scala** delle immagini

- è molto semplice da calcolare
- ha una bassa correlazione con la qualità visiva percepita dell'immagine
- è sensibile alla scala delle due immagini confrontate

Peak Signal to Noise Ratio (PSNR)

$$PSNR = 10\log_{10}\left(\frac{\max(x)^2}{MSE}\right)$$

→ viene misurato in Decibel (dB) e indica il rapporto tra il segnale e il rumore nell'immagine
→ il suo valore è solitamente compreso nell'intervallo [0, 100]: **maggiore** è il PSNR, **migliore** è l'immagine (i valori standard per immagini di buona qualità si attestano intorno a 30-40 dB)
→ è una misura standard nell'imaging che permette di confrontare immagini in contesti diversi poiché **non dipende dalla scala**

Structural Similarity Index (SSIM)

Non esiste una formula semplice per calcolare l'SSIM

- è più informativo del semplice errore numerico (maggiormente **correlato all'ispezione visiva** umana)
- è sensibile al **contrasto** e alla **struttura** dell'immagine

Altre metriche

Esistono altre metriche specifiche per determinate applicazioni → es. il rapporto contrasto-rumore **(Contrast-to-Noise-Ratio, CNR)** nell'imaging medico

X-Rays Computed Tomography

💡 La radiografia è una tecnica di imaging che utilizza i Raggi X (o radiazioni ionizzanti simili) per visualizzare la **struttura interna** (forma) di un oggetto

I Raggi X sono raggi ionizzanti → la capacità di questa radiazione di penetrare i tessuti molli, ma di essere bloccata dai materiali densi, ha avuto molte implicazioni in ambito medico, permettendo di osservare l'interno del corpo umano senza procedure invasive

Computed tomography

A differenza della radiografia tradizionale, che produce una singola proiezione 2D "schiacciando" tutte le strutture sovrapposte su un unico piano, la tomografia è l'imaging per sezioni (o "fette") → non si tratta di un'unica immagine complessiva, ma della ricostruzione di un singolo strato del corpo

L'acronimo italiano TAC sta per **Tomografia Assiale Computerizzata** (in inglese *Axial Computed Tomography* o semplicemente CT) → alla base di questa tecnologia c'è una **metodologia circolare** di acquisizione dei dati

L'acquisizione inizia con un tubo a raggi X che emette un fascio di radiazioni → questo fascio viene sagomato da un **collimatore** affinché assuma la forma di un "ventaglio" (fan beam) molto sottile, mirato esclusivamente alla sezione del corpo da analizzare.

I fotoni che compongono i raggi X vengono **parzialmente assorbiti** dai tessuti del corpo in base alla loro densità → quelli che riescono ad attraversare il corpo emergono dall'altro lato e vengono **registrati da una schiera di sensori (Detectors)**

I sensori inviano i dati registrati a un computer → attraverso algoritmi matematici, il computer calcola l'assorbimento punto per punto e ricostruisce l'immagine di quella specifica "fetta" trasversale, visualizzandola sul monitor

La peculiarità della TAC è che il tubo a raggi X e i sensori ad esso opposti **ruotano fisicamente attorno al paziente** (come indicato dalle frecce curve)

Third CT generation

Questa è l'architettura base dei sistemi moderni, in cui si elimina finalmente il movimento lineare

- **Geometria:** fascio a ventaglio ampio (*wide fan beam*). L'apertura è di **35-50 gradi**, sufficiente a "illuminare" l'intera larghezza del paziente in un solo istante
- **Sensore:** ampio arco curvo composto da **300-800 pixel**
- **Movimento ("Rotate-Rotate"):** la traslazione non serve più. Ora la sorgente e l'arco di rilevatori sono fissati rigidamente sui lati opposti e **ruotano insieme** attorno al paziente
- **Tempo:** circa **1 secondo** per una scansione completa

Per scansionare interi organi o corpi interi, è necessario muoversi anche lungo l'asse Z (longitudinale):

a. *Movimento discreto:* il paziente viene spostato a scatti (step-and-shoot). Si fa una fetta, si sposta il lettino, si fa un'altra fetta
b. *Traiettoria a Spirale/Elicoidale:* ruota continuamente mentre il lettino del paziente scorre in avanti a velocità costante. Questo permette di acquisire un volume continuo molto più velocemente

3D medical CT technique

Approccio standard per ottenere un modello tridimensionale dell'anatomia del paziente:

- il paziente viene scansionato da innumerevoli posizioni angolari
- nella sua forma "classica", questa tecnica impiega un'**alta dose di radiazioni X** per raccogliere un numero enorme di proiezioni e garantire la massima fedeltà dei dati

- il paziente giace su un lettino motorizzato che lo fa scorrere all'interno dell'anello, permettendo di scansionare progressivamente tutto il volume corporeo necessario

Matematicamente e computazionalmente, il volume 3D non è generato in un colpo solo: il computer ricostruisce una serie di singole fette 2D (le *slices*) ⇒ il modello tridimensionale è semplicemente **una pila di queste immagini 2D sovrapposte**

→ questo metodo tradizionale, proprio perché acquisisce un'alta densità di dati ad alta dose, produce immagini di **altissima qualità diagnostica**

Cone Beam Computed Tomography

Mentre la TAC medica classica ricostruisce il volume impilando fette 2D acquisite una alla volta (o a piccoli gruppi), la **Cone Beam Computed Tomography (CBCT)** acquisisce un intero volume 3D in un singolo giro

- I raggi X emessi dalla sorgente (*X-ray source*) divergono formando un **cono** tridimensionale, non più una linea o un ventaglio piatto (fan beam)
- Il sensore opposto alla sorgente è un pannello piatto bidimensionale e di grandi dimensioni

  → invece di registrare un profilo 1D per formare una fetta, cattura una vera e propria "fotografia" radiografica 2D ad ogni istante
- Il cono di raggi X avvolge interamente l'oggetto di interesse (*Object*) ⇒ ruotando attorno all'asse centrale per un solo giro (o una frazione di esso), il sistema raccoglie dati sufficienti per ricostruire direttamente l'intero volume 3D

Come collezionare i dati

La CT esegue una scansione circolare attorno al paziente, registrando un "profilo di attenuazione" da ogni angolazione → l'insieme di tutti questi profili acquisiti, mappati in un piano le cui coordinate sono lo spazio e l'angolo (t, θ), è chiamato **sinogramma:** rappresenta i dati "grezzi" (raw data) prima che vengano trasformati nell'immagine anatomica finale

Dal punto di vista matematico, il problema è descritto usando variabili continue:

- spazio continuo $t \in (-\infty, +\infty)$ → la posizione lungo la linea del rilevatore
- angolo continuo $\theta \in [0, 2\pi)$ → la rotazione completa a 360°

Poiché i computer e i sensori lavorano con dati finiti, nella realtà si opera in un dominio discreto (campionato):

- **Campionamento Angolare:** le proiezioni non sono continue, ma vengono calcolate da un numero finito di angoli $\theta_k \in \{\theta_1, \dots, \theta_{n_{angoli}}\}$
- **Campionamento Spaziale (Rilevatore):** i coefficienti di attenuazione non sono misurati su una linea continua, ma in un numero finito di punti discreti t_i , $\text{per ogni } i = 1, \dots, n_{pixel}$ (questo valore corrisponde alla risoluzione fisica del sensore)

  → es: si utilizza un rilevatore con 600 pixel (punti sensibili) per ricostruire un'immagine (o un oggetto) formata da una griglia di 512×512 voxel

→ l'insieme di questi campioni discreti, formalizzato matematicamente come $m(t_i, \theta_k) = P_{\theta_k}(t_i)$, costituisce il sinogramma digitale che l'algoritmo di ricostruzione dovrà invertire per ottenere l'immagine medica

Noisy sinogram

Il sinogramma reale acquisito dai sensori non corrisponde mai perfettamente al modello matematico delle proiezioni ideali → **rumore** (*noise*)

Due fonti principali di rumore nel processo di acquisizione CT:

1. **Rumore dalla Sorgente (Rumore di Poisson / Shot Noise):**

- *Causa fisica:* l'emissione dei fotoni a Raggi X dal tubo radiogeno non è un flusso perfettamente continuo e costante, ma è soggetta a fluttuazioni casuali (statistiche)
- *Modello matematico:* questo fenomeno legato al conteggio discreto dei fotoni viene descritto e modellizzato attraverso la **distribuzione di Poisson**

2. **Rumore dal Rilevatore (Rumore Gaussiano / White Noise):**
   - *Causa fisica:* l'elettronica del sensore e, soprattutto, il processo di conversione del segnale da analogico a digitale introducono disturbi elettrici
   - *Modello matematico:* Questo tipo di rumore è modellato come **rumore bianco** (*white noise*), tipicamente descritto da una **distribuzione Gaussiana a media nulla** (*zero mean gaussian distribution*)

The back projection

1. **Fase di Acquisizione**
   - al centro c'è un **oggetto** che rappresenta la sezione del paziente, al cui interno c'è un piccolo cerchio bianco che simula un dettaglio ad alta densità (es. un osso)
   - le frecce rette indicano i **fasci di raggi** X che attraversano l'oggetto da tre angolazioni differenti
   - **i** grafici (per ogni *view*) mostrano l'**attenuazione registrata** dai sensori per ogni vista ⇒ quando il raggio attraversa la zona grigia uniforme il segnale è basso e piatto, mentre quando attraversa il punto bianco denso l'attenuazione sale bruscamente, creando un "picco" nel grafico
2. **Fase di Ricostruzione**
   - l'algoritmo prende il profilo di attenuazione di ogni vista e lo "spalma" **(retroproietta)** su una griglia vuota, seguendo la direzione da cui era stato acquisito
   - nel punto esatto in cui i tre picchi si incrociano, **i valori si sommano** creando un punto bianco molto luminoso ⇒ l'algoritmo ha localizzato la posizione dell'oggetto

Guardando il risultato finale, si nota un problema matematico intrinseco di questo metodo: oltre al punto centrale luminoso, rimangono visibili le "**scie**" delle retroproiezioni ⇒ se usassimo centinaia di acquisizioni, queste scie formerebbero una **sfocatura** a raggiera attorno all'oggetto

→ per risolvere questo problema, la CT moderna non usa la retroproiezione semplice, ma la **Filtered Back Projection (FBP)**, che applica un filtro matematico (tipicamente un filtro *high-pass*) ai profili prima di retroproiettarli

Per eliminare la sfocatura, l'algoritmo non lavora più solo nello spazio visivo (dominio dell'immagine), ma passa al **dominio delle frequenze**:

- si applica la Trasformata di Fourier al sinogramma e si lavora sulle sue **componenti ad alta frequenza** (amplificare le alte frequenze significa "affilare" i bordi e sopprimere la sfocatura globale)
- una volta "filtrato" il segnale, si applica l'antitrasformata per tornare nel dominio dell'immagine spaziale

→ **vantaggio computazionale:** lavorando in Fourier, non è necessario discretizzare calcoli integrali complessi nel dominio spaziale, rendendo l'algoritmo matematicamente e computazionalmente più gestibile (alla base della *Filtered Back Projection*).

Safety in tomographic protocols

Nei protocolli di acquisizione TAC, la priorità è ridurre al minimo i rischi per la salute associati all'uso di **radiazioni ionizzanti** → per rispettare il principio ALARA (ridurre i rischi delle radiazioni), i protocolli TAC moderni adottano strategie che degradano volutamente l'acquisizione dei dati

- Nell'acquisizione ideale (Full-dose), il tubo radiogeno compie una traiettoria circolare completa sparando un'alta dose di raggi X per migliaia di angolazioni ravvicinate

→ il sinogramma generato è perfettamente fluido e continuo, la ricostruzione finale è nitida, contrastata e priva di difetti

Per abbassare le radiazioni, si può agire sull'intensità del raggio oppure sulla geometria di acquisizione:

1. **Low-dose CT (Riduzione dell'intensità)**

   → mantiene la geometria "piena", ma si abbassa drasticamente la potenza del raggio X per ogni singolo scatto

   → l'immagine finale è fortemente degradata da **noise** → *sfida computazionale: denoising senza sfocare i bordi veri*

2. **Riduzione del numero di scansioni (Geometria incompleta)**

   → invece di abbassare l'intensità, si spara a dose piena ma si fanno meno scatti: si copre l'intero angolo, ma si imposta un **grande passo angolare** tra le proiezioni

   → la mancanza di dati angolari genera **artefatti a striscia (streak artifacts)**, ovvero linee rette che attraversano l'immagine rovinando il contrasto → *sfida computazionale: interpolazione dei dati mancanti*

3. **Limited-angle CT / Tomosintesi (Angolo limitato)**

   → si mantiene un passo angolare fitto, ma si copre solo un **arco circolare limitato**, senza girare completamente attorno al paziente

   → causa una gravissima **distorsione geometrica**, in cui i bordi dell'oggetto paralleli ai raggi mancanti non vengono ricostruiti (nell'immagine, la parte superiore e inferiore dell'ovale appaiono sfumate o assenti) → *sfida computazionale: problema del "missing wedge" (cuneo mancante) in frequenza*

CT reconstruction as a linear inverse problem

- **Instabilità (Low-dose CT):** abbassando la dose, introduciamo rumore → questo rumore viene enormemente amplificato nella soluzione finale, rendendola instabile

- **Non-unicità (Sparse-view CT):** se mancano dei dati angolari (scansioni rade), mancano equazioni al nostro sistema → matematicamente, esistono **infinite soluzioni possibili** (infinite immagini) che potrebbero generare quello stesso sinogramma incompleto

→ inoltre, i metodi analitici tradizionali basati sulla FBP (*Filtered Back Projection*) non sono "buoni risolutori" per questi scenari degradati

- **Problema Diretto:** partiamo dall'oggetto reale (il paziente) e, attraverso il processo di acquisizione CT, otteniamo i dati di proiezione (il Sinogramma)
- **Problema Inverso:** partiamo dai dati misurati (il Sinogramma) e cerchiamo di risalire "all'indietro" per ricostruire l'immagine digitale originale

Per superare i limiti della FBP e risolvere un problema mal posto, si cambia completamente approccio matematico:

1. **Formulazione a Sistema Lineare (Model-based):** il problema non viene più trattato con integrali continui (Fourier), ma viene **discretizzato** in un gigantesco sistema algebrico lineare del tipo Ax = b (dove b è il sinogramma, x è l'immagine da trovare, e A è la matrice di sistema che modella la geometria della macchina)
2. **Informazioni a Priori (Prior object information):** poiché ci sono infinite soluzioni matematiche, dobbiamo dire al computer quale scegliere, inserendo delle informazioni "a priori" su come dovrebbe essere fatta un'immagine medica reale (es. "i tessuti interni hanno colori uniformi", "i bordi degli organi sono netti")
3. **Regolarizzazione:** è lo strumento matematico (es. Tikhonov, Total Variation) che "forza" il sistema lineare a trovare una soluzione **unica e accurata**, penalizzando le soluzioni che contengono troppo rumore o che non rispettano le nostre informazioni a priori

In pratica:

- la sorgente a raggi X (S) emette il fascio, che attraversa un'area quadrata per poi colpire i sensori del rilevatore $m_1, m_2, \ldots, m_7$
- l'area continua del corpo del paziente (una sezione di spessore δ) viene divisa in una griglia discreta di pixel/voxel di dimensione $N_x \times N_y$
- l'intera griglia viene "srotolata" in un unico vettore colonna di lunghezza $N = N_x \times N_y$, in cui ogni cella è un'incognita $x_j \ (\text{per } j = 1, \ldots, N)$

  → il valore $x_j \in \mathbb{R}$ che vogliamo scoprire non è altro che il **coefficiente di attenuazione locale** μ di quello specifico frammento di tessuto

$$\sum_{j=1}^{N} a_{i,j} x_j = m_i$$

→ questa singola equazione lineare descrive **un solo raggio:** ripetendo questa equazione per tutti i raggi sparati e per tutte le angolazioni θ, si costruisce il gigantesco sistema matriciale **Ax = m** (dove A contiene tutti i pesi geometrici $a_{i,j}$, x è l'immagine vettorizzata incognita, e m è il sinogramma misurato)

La matrice A è composta da milioni di coefficienti $a_{i,j}$ ⇒ *come calcolare i pesi* $a_{i,j}$ *?*

1. **Metodo "Ray driven" (Guidato dal raggio)**

   → si traccia una linea matematica dalla sorgente (Source) al singolo sensore (Pixel), seguendo esattamente il percorso del raggio X

   → il peso $a_{i,j}$ viene calcolato come il rapporto tra la lunghezza del segmento di raggio che cade all'interno del voxel in rosa e la lunghezza totale del raggio che attraversa l'oggetto (il segmento da A a B)

2. **Metodo "Pixel/Voxel driven" (Guidato dal voxel)**

   → invece di seguire il raggio, si prende in considerazione il singolo voxel e lo si "proietta" come un'ombra sul rilevatore

   → il peso $a_{i,j}$ si ottiene misurando l'area dell'"ombra" del voxel proiettata sul sensore divisa per l'area totale del sensore stesso